

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



SOICT

Báo Cáo Nhập môn An toàn thông tin

Đề tài: Các cơ chế phát hiện xâm nhập mạng

Học phần: Nhập môn An toàn thông tin

Mã học phần: IT4015

Giảng viên hướng dẫn: PGS. TS. Nguyễn Linh Giang

Sinh viên thực hiện:	Nguyễn Đình Tuấn Minh	20230048
	Vũ Đức Tâm	20230064
	Nguyễn Gia Khánh	202416803
	Nguyễn Đăng Cao Tuấn	202400119

Hà Nội, June 21, 2026

Mục lục

Lời nói đầu	3
1 Giới thiệu đề tài	4
1.1 Bối cảnh và lý do chọn đề tài	4
1.2 Mục tiêu đề tài	4
1.3 Đối tượng và phạm vi nghiên cứu	4
1.4 Phương pháp thực hiện	5
1.5 Bố cục báo cáo	5
2 Cơ sở lý thuyết	6
2.1 IDS, IPS và NIDS	6
2.2 Phát hiện theo chữ ký	7
2.3 Phát hiện theo hành vi	7
2.4 Các kiểu tấn công trong phạm vi đồ án	7
2.4.1 Quét cổng	7
2.4.2 ICMP Flood và SYN Flood	7
2.4.3 DNS độc hại và DNS Tunneling	8
2.4.4 Tấn công HTTP dựa trên chữ ký	8
2.5 Giới hạn lý thuyết liên quan đến triển khai	8
3 Kiến trúc và Triển khai Hệ thống NIDS	9
3.1 Tổng quan kiến trúc	9
3.2 Thu thập và chuẩn hóa gói tin	9
3.2.1 Capture	9
3.2.2 PacketEvent	10
3.3 Detection Engine	10
3.4 Luật phát hiện hành vi	11
3.5 Luật chữ ký kiểu Suricata	11
3.6 Lưu trữ và dashboard	11
4 Kịch bản Thử nghiệm và Đánh giá	13
4.1 Môi trường thử nghiệm	13
4.2 Khởi động hệ thống	13
4.3 Các kịch bản kiểm thử	14
4.3.1 Quét cổng bằng nmap	14
4.3.2 ICMP Ping Flood	14
4.3.3 TCP SYN Flood	14
4.3.4 DNS đáng ngờ và DNS tunneling	14
4.3.5 HTTP signature rules	14
4.4 Đánh giá kết quả	15

5	Kết luận và Hướng phát triển	16
5.1	Kết quả đạt được	16
5.2	Hạn chế	16
5.3	Hướng phát triển	17
5.4	Kết luận	17
	Tài liệu tham khảo	18

Lời nói đầu

Trong kỷ nguyên số hóa, hệ thống mạng là nền tảng cho hầu hết hoạt động của tổ chức và doanh nghiệp. Cùng với lợi ích đó, các rủi ro an ninh mạng cũng xuất hiện thường xuyên hơn, đặc biệt là các hành vi thăm dò, khai thác dịch vụ và truyền dữ liệu bất thường qua những giao thức hợp lệ.

Trong bối cảnh đó, các biện pháp phòng thủ tĩnh như tường lửa vẫn cần thiết nhưng chưa đủ để quan sát nội dung và hành vi của lưu lượng mạng. Hệ thống phát hiện xâm nhập mạng (NIDS) giúp bổ sung lớp giám sát, tạo cảnh báo khi lưu lượng có dấu hiệu bất thường hoặc khớp với mẫu tấn công đã biết.

Trong khuôn khổ học phần **IT4015 - Nhập môn An toàn thông tin**, nhóm em lựa chọn thực hiện đề tài: **"Các cơ chế phát hiện xâm nhập mạng"**. Trọng tâm của đề án là xây dựng một NIDS nhỏ gọn bằng Python để minh họa quy trình thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký và hiển thị cảnh báo.

Mục tiêu của báo cáo là trình bày ngắn gọn cơ sở lý thuyết cần thiết, mô tả kiến trúc hệ thống đã triển khai, kiểm thử trong môi trường lab và chỉ ra các giới hạn còn tồn tại.

Mặc dù đã nỗ lực tìm hiểu và tổng hợp tài liệu, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, bản báo cáo chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự nhận xét và góp ý quý báu từ thầy để kiến thức của mình được hoàn thiện hơn. Em xin chân thành cảm ơn!

Hà Nội, ngày ... tháng 04 năm 2026

Nhóm sinh viên thực hiện

Chương 1

Giới thiệu đề tài

1.1 Bối cảnh và lý do chọn đề tài

Trong các hệ thống mạng hiện đại, tường lửa chỉ là một lớp bảo vệ ban đầu. Nhiều cuộc tấn công vẫn đi qua các cổng hợp lệ như HTTP, DNS hoặc HTTPS, sau đó khai thác điểm yếu ở tầng ứng dụng hoặc tạo ra các hành vi bất thường trong mạng nội bộ. Vì vậy, hệ thống phát hiện xâm nhập mạng (Network Intrusion Detection System – NIDS) là một thành phần quan trọng để giám sát lưu lượng, phát hiện dấu hiệu tấn công và hỗ trợ quản trị viên phản ứng kịp thời.

Đề tài này tập trung xây dựng một NIDS nhỏ gọn bằng Python nhằm minh họa các thành phần cốt lõi của một hệ thống phát hiện xâm nhập: thu thập gói tin, chuẩn hóa dữ liệu, phát hiện theo hành vi, phát hiện theo chữ ký, lưu trữ cảnh báo và hiển thị qua dashboard. Mục tiêu chính là phục vụ học tập và trình diễn kỹ thuật, không hướng tới thay thế các hệ thống sản xuất như Suricata hoặc Zeek.

1.2 Mục tiêu đề tài

Đề án đặt ra các mục tiêu sau:

- **Thu thập và chuẩn hóa gói tin:** đọc lưu lượng từ tệp PCAP hoặc giao diện mạng trực tiếp bằng Scapy, sau đó chuyển đổi thành cấu trúc sự kiện thống nhất.
- **Phát hiện theo hành vi:** triển khai các luật dựa trên cửa sổ thời gian để nhận diện quét cổng, ICMP flood, TCP SYN flood, truy vấn DNS đáng ngờ và dấu hiệu DNS tunneling.
- **Phát hiện theo chữ ký:** xây dựng bộ phân tích cú pháp cho một tập con của luật Suricata, hỗ trợ các trường payload, HTTP và DNS thường dùng trong demo.
- **Ghi nhận và trực quan hóa cảnh báo:** lưu cảnh báo dưới dạng JSON Lines và hiển thị thống kê qua giao diện Flask.
- **Kiểm thử trong môi trường lab:** mô phỏng các luồng tấn công phổ biến bằng Kali Linux để đánh giá khả năng kích hoạt cảnh báo của hệ thống.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là hệ thống phát hiện xâm nhập mạng dạng thụ động. Hệ thống quan sát lưu lượng, tạo cảnh báo và phục vụ phân tích, nhưng không trực tiếp chặn gói tin hoặc ngắt phiên kết nối như một hệ thống IPS.

Phạm vi triển khai của đề án được giới hạn như sau:

- Hệ thống xử lý các trường thông tin phổ biến của IP, TCP, UDP, ICMP, DNS và HTTP.
- Phần phát hiện hành vi sử dụng ngưỡng tĩnh và cửa sổ trượt thời gian, chưa áp dụng học máy.
- Phần chữ ký hỗ trợ một tập con cú pháp Suricata, đủ cho các tình huống demo như SQL Injection và nhận diện `sqlmap`.
- Hệ thống chưa giải mã TLS/HTTPS, chưa tái tạo đầy đủ TCP stream và chưa được tối ưu cho lưu lượng mạng tốc độ cao.

1.4 Phương pháp thực hiện

Đề tài được triển khai theo hướng kết hợp lý thuyết và thực nghiệm:

- Nghiên cứu các cơ chế IDS cơ bản, đặc biệt là phát hiện theo chữ ký và phát hiện theo hành vi.
- Thiết kế kiến trúc pipeline gồm các khối capture, parser, detection engine, storage và dashboard.
- Cài đặt từng module bằng Python, sử dụng Scapy cho xử lý gói tin và Flask cho giao diện web.
- Viết luật kiểm thử và kịch bản tấn công mô phỏng để kiểm tra kết quả cảnh báo.
- Đối chiếu kết quả với phạm vi đã đặt ra, đồng thời chỉ ra hạn chế còn tồn tại.

1.5 Bố cục báo cáo

Báo cáo gồm 5 chương:

- **Chương 1: Giới thiệu đề tài** – trình bày bối cảnh, mục tiêu, phạm vi và phương pháp thực hiện.
- **Chương 2: Cơ sở lý thuyết** – tóm tắt các khái niệm IDS/IPS, NIDS, phát hiện theo chữ ký và phát hiện theo hành vi.
- **Chương 3: Kiến trúc và triển khai hệ thống** – mô tả các module chính và cách dữ liệu đi qua hệ thống.
- **Chương 4: Kịch bản thử nghiệm và đánh giá** – trình bày môi trường lab, các kịch bản tấn công và kết quả quan sát.
- **Chương 5: Kết luận và hướng phát triển** – tổng kết kết quả đạt được, hạn chế và hướng cải tiến.

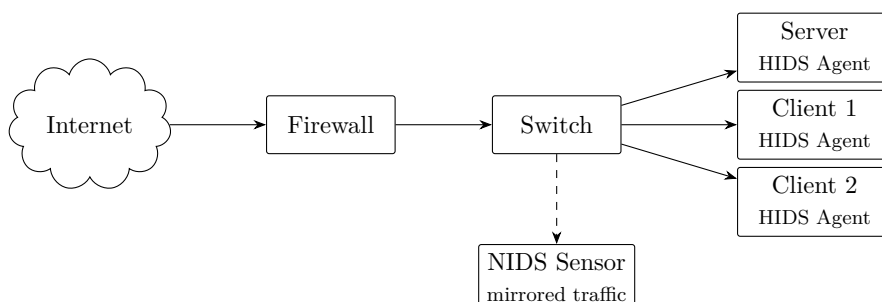
Chương 2

Cơ sở lý thuyết

2.1 IDS, IPS và NIDS

Hệ thống phát hiện xâm nhập (Intrusion Detection System – IDS) có nhiệm vụ thu thập và phân tích sự kiện bảo mật để phát hiện hành vi bất thường hoặc dấu hiệu tấn công. IDS thường hoạt động ở chế độ thụ động: hệ thống quan sát lưu lượng, ghi log và sinh cảnh báo, nhưng không trực tiếp can thiệp vào luồng dữ liệu.

Hệ thống ngăn chặn xâm nhập (Intrusion Prevention System – IPS) có phạm vi rộng hơn IDS vì được đặt inline trên đường truyền và có khả năng chặn gói tin, reset kết nối hoặc cập nhật luật tường lửa. Đồ án này chỉ triển khai hướng IDS, tức là tập trung vào phát hiện và cảnh báo. Theo nguồn dữ liệu giám sát, IDS có thể được chia thành hai nhóm chính:



Hình 2.1: Minh họa vị trí giám sát của NIDS và HIDS

Loại hệ thống	Nguồn dữ liệu	Đặc điểm
NIDS	Lưu lượng mạng tại switch, gateway hoặc interface giám sát.	Quan sát được nhiều máy trong cùng phân đoạn mạng, phù hợp phát hiện scan, flood và payload độc hại trong lưu lượng không mã hóa.
HIDS	Log, tiến trình, file và hành vi trên từng máy chủ hoặc endpoint.	Nhìn sâu vào trạng thái máy trạm nhưng cần cài agent trên từng host.

Bảng 2.1: Phân loại IDS theo nguồn dữ liệu giám sát

Hệ thống trong đồ án là một NIDS dạng thụ động: đọc PCAP hoặc sniff interface, phân tích gói tin và ghi cảnh báo ra tệp JSONL.

2.2 Phát hiện theo chữ ký

Phát hiện theo chữ ký (Signature-based Detection) so khớp lưu lượng với các mẫu đã biết. Mẫu có thể là chuỗi trong payload, biểu thức chính quy, trường HTTP, truy vấn DNS hoặc tổ hợp điều kiện trong header gói tin. Cách tiếp cận này phù hợp để nhận diện các kỹ thuật tấn công đã biết như SQL Injection, công cụ quét tự động hoặc payload có cấu trúc đặc trưng.

Một luật Suricata thường gồm hai phần:

```
alert http any any -> any 80 (msg:"sqlmap User-Agent";  
http.user_agent; content:"sqlmap"; nocase; sid:1000002; rev:1;)
```

- **Header luật:** xác định hành động, giao thức, địa chỉ IP, cổng và hướng lưu lượng.
- **Options:** mô tả điều kiện cần so khớp như `content`, `pcre`, `http.uri`, `http.user_agent`, `sid`, `priority`.

Ưu điểm của phát hiện theo chữ ký là cảnh báo rõ ràng và dễ kiểm chứng. Hạn chế là hệ thống chỉ phát hiện tốt các mẫu đã được mô tả trước; nếu payload bị mã hóa, bị làm rối hoặc chưa có rule phù hợp thì khả năng bỏ sót sẽ tăng.

2.3 Phát hiện theo hành vi

Phát hiện theo hành vi (Behavior-based Detection) quan sát chuỗi sự kiện theo thời gian thay vì chỉ xét một gói tin riêng lẻ. Trong đồ án này, cơ chế hành vi được triển khai bằng cửa sổ trượt thời gian. Hệ thống lưu các sự kiện gần nhất theo từng địa chỉ nguồn, loại bỏ các sự kiện đã quá hạn và kích hoạt cảnh báo khi số lượng hoặc đặc điểm sự kiện vượt ngưỡng.

Các ví dụ được triển khai:

- Một IP truy cập nhiều cổng TCP khác nhau trong thời gian ngắn có thể là quét cổng.
- Một IP gửi quá nhiều ICMP Echo Request có thể là ping flood.
- Một IP gửi quá nhiều gói TCP SYN có thể là SYN flood.
- Các truy vấn DNS có nhãn quá dài hoặc entropy cao có thể là dấu hiệu DNS tunneling.

Cách tiếp cận này không cần biết trước chính xác payload tấn công, nhưng phụ thuộc nhiều vào ngưỡng cấu hình. Ngưỡng quá thấp dễ tạo cảnh báo sai; ngưỡng quá cao có thể bỏ sót các cuộc tấn công chậm.

2.4 Các kiểu tấn công trong phạm vi đồ án

Báo cáo tập trung vào các tình huống có thể kiểm thử trực tiếp bằng hệ thống đã xây dựng.

2.4.1 Quét cổng

Quét cổng là bước thăm dò nhằm xác định dịch vụ đang mở trên máy mục tiêu. Dấu hiệu chính trong hệ thống là một địa chỉ nguồn gửi gói TCP tới nhiều cổng đích khác nhau trong cùng một cửa sổ thời gian.

2.4.2 ICMP Flood và SYN Flood

ICMP Flood tạo lượng lớn gói Echo Request để tiêu tốn băng thông hoặc tài nguyên xử lý. SYN Flood tạo nhiều yêu cầu TCP SYN chưa hoàn tất kết nối. Cả hai đều được phát hiện bằng bộ đếm theo nguồn trong cửa sổ thời gian.

2.4.3 DNS độc hại và DNS Tunneling

DNS có thể bị lợi dụng để truy vấn tên miền độc hại hoặc truyền dữ liệu ra ngoài thông qua các nhãn dài, khó đọc, entropy cao. Hệ thống kiểm tra tên miền đáng ngờ và thống kê số lượng truy vấn có đặc điểm bất thường.

2.4.4 Tấn công HTTP dựa trên chữ ký

Một số payload tấn công web có thể được nhận diện bằng nội dung trong URI, User-Agent hoặc body HTTP. Demo sử dụng luật tùy chỉnh để phát hiện chuỗi SQL Injection cơ bản và User-Agent của sqlmap.

2.5 Giới hạn lý thuyết liên quan đến triển khai

Các cơ chế trên có một số giới hạn cần được hiểu đúng khi đánh giá kết quả:

- NIDS không nhìn thấy nội dung ứng dụng nếu lưu lượng được mã hóa bằng TLS mà không có cơ chế giải mã hợp lệ.
- Nếu không tái tạo TCP stream, chữ ký có thể bỏ sót payload bị chia nhỏ qua nhiều gói tin.
- Cảnh báo hành vi phụ thuộc vào ngưỡng và môi trường mạng cụ thể.
- Hệ thống demo dùng Python và Scapy phù hợp cho học tập, nhưng không đại diện cho hiệu năng của IDS triển khai trong mạng doanh nghiệp.

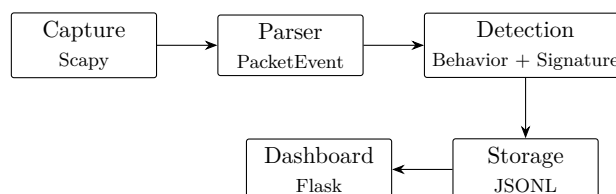
Chương 3

Kiến trúc và Triển khai Hệ thống NIDS

3.1 Tổng quan kiến trúc

Hệ thống NIDS được xây dựng bằng Python 3 và tổ chức theo mô hình pipeline. Mỗi gói tin sau khi được đọc từ PCAP hoặc interface mạng sẽ đi qua các bước: thu thập, chuẩn hóa, phát hiện, lưu trữ và hiển thị.

1. **Capture:** đọc gói tin bằng Scapy từ tệp PCAP hoặc giao diện mạng.
2. **Parser:** chuyển gói tin Scapy thành đối tượng `PacketEvent`.
3. **Detection Engine:** chạy luật hành vi và luật chữ ký.
4. **Storage:** ghi cảnh báo ra tệp JSONL.
5. **Dashboard:** đọc dữ liệu cảnh báo và hiển thị thống kê bằng Flask.



Hình 3.1: Luồng dữ liệu của hệ thống NIDS

3.2 Thu thập và chuẩn hóa gói tin

3.2.1 Capture

Module `nids/capture.py` cung cấp hai cách lấy dữ liệu: đọc từ tệp PCAP và sniff trực tiếp trên interface. Với live capture, Scapy gọi callback cho từng gói tin; nếu parser chuyển đổi thành công thì sự kiện sẽ được đưa vào detection engine.

Đoạn mã 3.1: Live capture bằng Scapy

```
1 def sniff_live(interface: str, callback: PacketCallback, limit: int = 0) -> None:
2     from scapy.all import sniff
3
```

```
4 def handle_packet(packet: object) -> None:
5     event = packet_to_event(packet)
6     if event is not None:
7         callback(event)
8
9     sniff(iface=interface, prn=handle_packet, store=False, count=limit)
```

3.2.2 PacketEvent

Raw packet chứa nhiều trường phụ thuộc giao thức nên khó xử lý trực tiếp trong rule engine. Vì vậy, module `nids/parser.py` chuẩn hóa các trường cần thiết thành `PacketEvent`: thời gian, IP nguồn/đích, cổng nguồn/đích, giao thức, TCP flags, ICMP type, DNS query, payload text và các trường HTTP phổ biến.

Đoạn mã 3.2: Cấu trúc PacketEvent

```
1 @dataclass(frozen=True)
2 class PacketEvent:
3     timestamp: float
4     src_ip: str | None
5     dst_ip: str | None
6     src_port: int | None
7     dst_port: int | None
8     protocol: str
9     length: int
10    tcp_flags: str | None = None
11    icmp_type: int | None = None
12    dns_query: str | None = None
13    payload_text: str | None = None
14    http_method: str | None = None
15    http_uri: str | None = None
16    http_host: str | None = None
17    http_user_agent: str | None = None
```

Payload được giới hạn kích thước khi chuyển thành chuỗi để tránh tốn bộ nhớ và giảm chi phí so khớp. Đây là lựa chọn phù hợp cho demo, nhưng cũng là một giới hạn khi phân tích payload lớn hoặc payload bị chia nhỏ.

3.3 Detection Engine

Module `nids/engine.py` đóng vai trò điều phối. Mỗi `PacketEvent` được đếm vào thống kê giao thức, sau đó gửi đến `SlidingWindowRules`. Các cảnh báo sinh ra được ghi xuống `AlertStore`.

Đoạn mã 3.3: Luồng xử lý một sự kiện

```
1 def process_event(self, event: PacketEvent) -> list[Alert]:
2     self.packet_count += 1
3     self.protocol_counts[event.protocol] += 1
4
5     alerts = self.rules.evaluate(event)
6     self.store.append_many(alerts)
7     self.alert_count += len(alerts)
8     return alerts
```

3.4 Luật phát hiện hành vi

Các luật hành vi được triển khai bằng cửa sổ trượt theo thời gian. Hệ thống lưu các sự kiện gần nhất theo IP nguồn và loại bỏ dữ liệu đã nằm ngoài khung thời gian cấu hình.

Đoạn mã 3.4: Loại bỏ sự kiện cũ trong cửa sổ thời gian

```
1 @staticmethod
2 def _drop_old_times(window: deque[float], now: float, seconds: int) -> None:
3     while window and now - window[0] > seconds:
4         window.popleft()
```

Các luật tích hợp gồm:

Rule ID	Tên luật	Điều kiện chính
RULE-001	Port Scan	Một nguồn truy cập 20 cổng TCP đích khác nhau trong 10 giây.
RULE-002	ICMP Ping Flood	Một nguồn gửi 100 ICMP Echo Request trong 10 giây.
RULE-003	TCP SYN Flood	Một nguồn gửi 100 gói TCP SYN trong 10 giây.
RULE-004	Suspicious DNS	Truy vấn khớp tên miền đáng ngờ đã cấu hình.
RULE-005	DNS Tunneling	Nhiều truy vấn DNS có nhãn dài hoặc entropy cao trong 30 giây.

Bảng 3.1: Các luật hành vi tích hợp

3.5 Luật chữ ký kiểu Suricata

Ngoài luật hành vi, hệ thống hỗ trợ một tập con cú pháp Suricata. Người dùng có thể truyền tệp luật qua tham số --rules. Trong demo, tệp custom.rules chứa ba luật HTTP:

Đoạn mã 3.5: Luật chữ ký dùng trong demo

```
1 alert http any any -> any 80 (msg:"HTTP URI SQL Injection Attempt"; http.uri;
   content:"' OR '1'='1"; nocase; classtype:web-application-attack; priority:1; sid
   :1000001; rev:1;)
2
3 alert http any any -> any 80 (msg:"sqlmap Scanner User-Agent Detected"; http.
   user_agent; content:"sqlmap"; nocase; classtype:attempted-recon; priority:2; sid
   :1000002; rev:1;)
4
5 alert tcp any any -> any 80 (msg:"HTTP POST Body SQL Injection Attempt"; pkt_data;
   content:"' OR '1'='1"; nocase; classtype:web-application-attack; priority:1; sid
   :1000003; rev:1;)
```

Mục tiêu của phần này là minh họa cách một signature engine hoạt động: phân tích header luật, chọn buffer cần kiểm tra, sau đó so khớp nội dung hoặc biểu thức chính quy.

3.6 Lưu trữ và dashboard

Cảnh báo được biểu diễn bằng Alert và ghi dưới dạng JSON Lines. Mỗi dòng là một cảnh báo độc lập nên hệ thống có thể append liên tục mà không cần đọc lại toàn bộ tệp.

Dashboard trong `dashboard/app.py` đọc tệp cảnh báo, cache theo thời gian sửa đổi của file và hiển thị:

- tổng số cảnh báo,
- số cảnh báo theo mức độ nghiêm trọng,
- các IP nguồn xuất hiện nhiều nhất,
- danh sách cảnh báo gần nhất.

Dashboard phục vụ quan sát và trình diễn kết quả; phần phản ứng tự động như chặn IP hoặc cập nhật tường lửa chưa nằm trong phạm vi đề án.

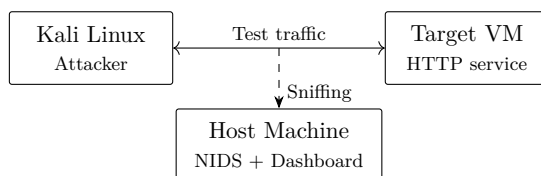
Chương 4

Kịch bản Thử nghiệm và Đánh giá

4.1 Môi trường thử nghiệm

Việc kiểm thử được thực hiện trong môi trường lab ảo hóa để tránh ảnh hưởng đến mạng thật. Mô hình gồm ba thành phần:

- **Host Machine:** chạy NIDS engine và Flask dashboard.
- **Kali Linux VM:** đóng vai trò máy tạo lưu lượng kiểm thử bằng `nmap`, `hping3`, `dig` và `curl`.
- **Target VM:** nhận lưu lượng scan, flood, DNS và HTTP. Target cần có dịch vụ HTTP ở cổng 80 để kiểm thử luật chữ ký web.



Hình 4.1: Mô hình lab kiểm thử

4.2 Khởi động hệ thống

Trên host, dashboard được chạy ở một terminal:

Đoạn mã 4.1: Khởi động dashboard

```
1 python3 dashboard/app.py
```

NIDS engine được chạy ở terminal khác. Interface cần thay bằng interface thực tế kết nối các máy ảo:

Đoạn mã 4.2: Khởi động NIDS engine

```
1 sudo .venv/bin/python main.py --iface <YOUR_INTERFACE> --rules custom.rules --limit 0
```

Khi dùng PCAP thay cho live capture, có thể chạy:

Đoạn mã 4.3: Phân tích PCAP

```
1 python3 main.py --pcap sample.pcap --rules custom.rules
```

4.3 Các kịch bản kiểm thử

4.3.1 Quét cổng bằng nmap

Kịch bản sử dụng **nmap** để gửi SYN tới nhiều cổng đích trong thời gian ngắn:

```
1 nmap -sS -p 21,22,23,25,53,80,110,135,139,143,443,445,\
2 993,995,1433,1723,3306,3389,5900,8080,8443,8888,9090,\
3 9200,9300 --min-rate 300 -T4 -n --open <TARGET_IP>
```

Kết quả mong đợi là cảnh báo **RULE-001 Port Scan**, vì cùng một IP nguồn truy cập nhiều cổng TCP đích trong cửa sổ 10 giây.

4.3.2 ICMP Ping Flood

Kịch bản gửi nhiều gói ICMP Echo Request:

```
1 hping3 --icmp -c 120 -i u5000 <TARGET_IP>
```

Kết quả mong đợi là cảnh báo **RULE-002 ICMP Ping Flood** khi bộ đếm ICMP từ cùng nguồn vượt ngưỡng.

4.3.3 TCP SYN Flood

Kịch bản gửi nhiều gói SYN tới cổng 80:

```
1 hping3 -S -p 80 -c 120 -i u5000 <TARGET_IP>
```

Kết quả mong đợi là cảnh báo **RULE-003 TCP SYN Flood**. Trong phạm vi đề án, hệ thống chỉ cảnh báo dựa trên số lượng gói SYN, chưa kiểm tra trạng thái hoàn tất kết nối ở mức TCP stream đầy đủ.

4.3.4 DNS đáng ngờ và DNS tunneling

Kịch bản DNS gồm hai nhóm: truy vấn tên miền nằm trong danh sách đáng ngờ và truy vấn có nhãn dài/entropy cao.

```
1 dig malware.test
2 dig aGVsbG93b3JzZGh1bGxvd2...tunnel.example.com
```

Kết quả mong đợi là **RULE-004 Suspicious DNS Query** với tên miền đáng ngờ và **RULE-005 DNS Tunneling Suspicion** khi có đủ số truy vấn bất thường trong cửa sổ thời gian.

4.3.5 HTTP signature rules

Các request HTTP được dùng để kiểm tra luật chữ ký:

```
1 curl -s "http://<TARGET_IP>/index.php?id=%27%20OR%20%27%27%3D%27"
2 curl -A "sqlmap/1.7.8#stable" "http://<TARGET_IP>/"
3 curl -s -X POST "http://<TARGET_IP>/login" \
4 --data "username=admin&password=%27+OR+%27%27%3D%27"
```

Kết quả mong đợi là các cảnh báo từ luật `custom.rules`: SQL Injection trong URI, User-Agent chứa `sqlmap`, và SQL Injection trong body POST.

4.4 Đánh giá kết quả

Trong môi trường lab, các kịch bản trên kiểm tra được cả hai lớp phát hiện:

- Luật hành vi phát hiện các mẫu dựa trên tần suất và số lượng sự kiện trong thời gian ngắn.
- Luật chữ ký phát hiện các chuỗi đặc trưng trong HTTP và payload thô.
- Dashboard hiển thị được số lượng cảnh báo, mức độ nghiêm trọng, IP nguồn và danh sách cảnh báo gần nhất.

Kết quả này xác nhận hệ thống đáp ứng mục tiêu học tập và trình diễn. Tuy nhiên, đánh giá này chỉ có giá trị trong phạm vi lab có kiểm soát. Để kết luận về độ chính xác trong mạng thực tế cần có bộ dữ liệu lớn hơn, lưu lượng nền đa dạng hơn, kiểm thử false positive/false negative và đo hiệu năng xử lý gói tin.

Chương 5

Kết luận và Hướng phát triển

5.1 Kết quả đạt được

Đồ án đã xây dựng được một hệ thống NIDS nhỏ gọn phục vụ học tập và trình diễn. Hệ thống hoàn chỉnh các thành phần chính của một pipeline phát hiện xâm nhập mạng:

- Đọc lưu lượng từ PCAP hoặc live interface bằng Scapy.
- Chuẩn hóa packet thành cấu trúc `PacketEvent`.
- Phát hiện hành vi bằng cửa sổ trượt thời gian.
- Phân tích một tập con luật Suricata để phát hiện chữ ký trong HTTP, DNS và payload.
- Ghi cảnh báo ra JSON Lines và hiển thị qua dashboard Flask.
- Có kịch bản demo để kiểm tra các rule chính trong môi trường lab.

Kết quả quan trọng nhất của đồ án là làm rõ cách các thành phần IDS phối hợp với nhau: capture cung cấp dữ liệu, parser chuẩn hóa dữ liệu, rule engine tạo cảnh báo, storage lưu lại kết quả và dashboard hỗ trợ quan sát.

5.2 Hạn chế

Hệ thống hiện tại vẫn có một số hạn chế kỹ thuật:

- **Chỉ phát hiện, chưa ngăn chặn:** hệ thống không chặn gói tin, không reset kết nối và không tự động cập nhật tường lửa.
- **Chưa xử lý lưu lượng mã hóa:** nội dung HTTPS/TLS không thể được kiểm tra bằng chữ ký nếu không có cơ chế giải mã hợp lệ.
- **Chưa tái tạo TCP stream đầy đủ:** payload bị chia nhỏ qua nhiều gói tin có thể làm giảm hiệu quả của signature matching.
- **Hiệu năng còn giới hạn:** Python và Scapy phù hợp cho demo, nhưng chưa phù hợp với môi trường có lưu lượng lớn nếu không tối ưu thêm.
- **Phụ thuộc vào ngưỡng và luật:** chất lượng cảnh báo phụ thuộc vào cấu hình threshold và độ đầy đủ của tập luật chữ ký.

5.3 Hướng phát triển

Các hướng cải tiến phù hợp với kiến trúc hiện tại gồm:

- Bổ sung TCP stream reassembly để kiểm tra payload ở mức phiên thay vì từng gói riêng lẻ.
- Mở rộng parser HTTP/DNS và tăng mức tương thích với cú pháp Suricata.
- Thêm cơ chế cấu hình rule và threshold qua tệp YAML hoặc giao diện dashboard.
- Đo hiệu năng trên PCAP lớn và tối ưu luồng xử lý để giảm packet drop.
- Tích hợp phản ứng bán tự động như xuất IOC, webhook hoặc cập nhật danh sách chặn cho firewall trong môi trường lab.
- Xây dựng bộ kiểm thử với cả traffic bình thường và traffic tấn công để đánh giá false positive/false negative rõ ràng hơn.

5.4 Kết luận

Hệ thống đã đáp ứng mục tiêu chính của đề án: minh họa được một NIDS hoạt động từ khâu thu thập gói tin đến sinh cảnh báo và trực quan hóa kết quả. Dù chưa phải hệ thống production, đề án cung cấp nền tảng thực hành tốt để hiểu các kỹ thuật phát hiện xâm nhập mạng và các giới hạn khi triển khai IDS trong thực tế.

Tài liệu tham khảo

Bibliography

- [1] Bộ môn Truyền thông và Mạng máy tính. *Bài giảng Nhập môn An toàn thông tin (IT4015)*.
- [2] William Stallings (2016). *Cryptography and Network Security: Principles and Practice* (7th Edition). Pearson Education.
- [3] James P. Anderson (1980). *Computer Security Threat Monitoring and Surveillance*. Technical report, James P. Anderson Co., Fort Washington, Pennsylvania.
- [4] Dorothy E. Denning (1987). *An Intrusion-Detection Model*. IEEE Transactions on Software Engineering, Vol. SE-13, No. 2, pp. 222-232.
- [5] Karen Scarfone, Peter Mell (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication 800-94.
- [6] OISF. *Suricata User Guide*. Truy cập trực tuyến tại: <https://suricata.readthedocs.io/>.